

Escapa Buraco — Firmware v0.9

Guia para **gravar / atualizar o firmware** do jogo **Escapa Buraco** num **Arduino Uno + CNC Shield v3**, e também para quem quer **montar o próprio jogo em casa**.

- Só quer **atualizar o firmware**? Vá direto para **5. Software** e **8. Upload**.
- Vai **montar do zero**? Comece pela **2. Hardware**.

*Obs.: embora use um CNC Shield, este projeto **não usa GRBL**. É um firmware próprio que aproveita o shield apenas como interface dos motores — não grave GRBL na placa.*

Autoria e licença

Autoria: Fabricio Masutti e Leonardo Gallep — **Coletivo Máquina Tudo**

© 2026. Este projeto (código e documentação) é distribuído sob a licença **Creative Commons Atribuição-NãoComercial 4.0 Internacional (CC BY-NC 4.0)**.
<https://creativecommons.org/licenses/by-nc/4.0/deed.pt-br>

Você pode **compartilhar** e **adaptar** este material, desde que **dê o devido crédito** aos autores e **não o utilize para fins comerciais**.

*As bibliotecas de terceiros incluídas mantêm suas próprias licenças e **não** são cobertas por esta licença CC (ver Créditos no fim).*

1. Conteúdo do zip

```
EscapaBuraco/
  EscapeBuracoAnalog_v9/      pasta do sketch
    EscapeBuracoAnalog_v9.ino (programa principal)
    tocarMusica.ino           (melodias / sinais sonoros)
    pitches.h                  (notas musicais)
    pitches.cpp
  libraries/
```

TM1637-avishorp.zip	biblioteca do display (instalar 1 vez)
Imagens/	
Pinagem CNC Shield.jpeg	imagem de referência para CNC Shield
Pinagem Arduino CNC Shield.jpeg	imagem de referência para pinagem Ardu
LEIAME.md	este arquivo

Importante: o Arduino exige que o `.ino` principal esteja dentro de uma pasta com o **mesmo nome** dele. Por isso os 4 arquivos do sketch ficam juntos em `EscapeBuracoAnalog_v9/`. Ao descompactar, **não separe** esses arquivos nem renomeie a pasta.

2. Hardware: componentes e ferramentas

Componentes principais

Item	Descrição
Arduino Uno	placa controladora
CNC Shield v3	encaixa sobre o Uno
2× drivers de passo	DRV8825 (usado neste projeto) ou A4988 — apenas 2
2× dissipadores	heatsink adesivo, um por driver
2× motores de passo	NEMA 17 (42 mm)
Display TM1637	4 dígitos, mostra o tempo
Alto-falante pequeno	sinais sonoros (opção recomendada; ou buzzer passivo)
2× joysticks (microswitch)	controle das pontas (usa só sobe/desce)
2× fins de curso	referência de “zero” dos eixos X e Y
3× chaves	início, fim de percurso e buraco 10
Fonte externa	12 V (ver seção 4)
Cabo USB A-B	para gravar o firmware

Ferramentas necessárias

- Ferro de solda (para soldar os fios no display — ver seção 4).

- Multímetro (ajuste do Vref e identificação das bobinas).
 - Chave de fenda pequena (potenciômetro dos drivers).
 - Chave Philips.
 - Alicate pequeno meia-cana (ótimo para conectores DuPont e porcas de 3 mm).
-

3. Drivers de passo (micropasso, corrente, montagem)

3.1 Quantos e onde

Use **apenas 2 drivers**, encaixados nos soquetes **X** e **Y**. Deixe os soquetes **Z** e **A** vazios.

3.2 Montagem (atenção!)

- **Orientação:** encaixe o driver alinhando o pino **EN** do driver ao **EN/GND** marcado no shield. **Invertido, o driver queima ao energizar.**
- **Heatsink:** cole o dissipador adesivo no CI de cada driver.

3.3 Micropasso (jumpers M0/M1/M2 sob cada driver)

Os mesmos três jumpers significam coisas diferentes em cada driver
(= jumper instalado/fechado; — = sem jumper):

DRV8825

Resolução	M0	M1	M2
Passo completo	—	—	—
1/2		—	—
1/4	—		—
1/8			—
1/16	—	—	
1/32			

A4988

Resolução	MS1	MS2	MS3
-----------	-----	-----	-----

Passo completo	—	—	—
1/2		—	—
1/4	—		—
1/8			—
1/16			

Neste projeto: DRV8825 com os três jumpers fechados 1/32.

Repare que “todos fechados” dá 1/32 no DRV8825 e 1/16 no A4988 (que é o máximo dele).

***Nota sobre o firmware:** a constante `PASSOS_POR_REVOLUCAO = 3200` equivale a 1/16. Rodando em 1/32, ela funciona na prática como um **fator de escala** dos limites (o jogo foi calibrado assim). Se você montar com **outro driver ou micropasso**, o curso da barra muda na mesma proporção e os limites (`MAX_PONTOS` , o recuo de 700 passos, `MAX_DIFERENCA_PASSOS`) podem precisar de reajuste.*

3.4 Regulagem de corrente (Vref) — obrigatória

Cada driver precisa ter a corrente ajustada para os **seus** motores. Corrente alta demais superaquece; baixa demais perde passos.

1. Pegue a **corrente nominal por fase (I)** no **datasheet do motor** (NEMA 17 típico: ~1,2–1,7 A).
2. Descubra o **Rsense** da sua placa — são os resistores retangulares pequenos perto do CI do driver. O texto indica o valor: `R100 = 0,1 Ω`, `R050 = 0,05 Ω`, `R200 = 0,2 Ω`. **Confira na sua placa, pois varia entre fabricantes.**
3. Aplique a fórmula:
 - **DRV8825:** $V_{ref} = I \times 5 \times R_{sense}$ com $R_{sense} 0,1 \Omega$: **$V_{ref} = I \div 2$**
 - **A4988:** $V_{ref} = I \times 8 \times R_{sense}$ com $R_{sense} 0,1 \Omega$: **$V_{ref} = I \times 0,8$**
4. **Exemplo** (motor de 1,5 A, $R_{sense} 0,1 \Omega$):
 - DRV8825 $V_{ref} \approx$ **0,75 V**
 - A4988 $V_{ref} \approx$ **1,20 V**
5. **Como medir:** com a placa **energizada** (lógica), encoste uma ponta do multímetro no parafuso do potenciômetro e a outra no GND; ajuste com a chave de fenda pequena até o valor calculado. Comece um pouco **abaixo** do nominal e suba se perder passos. O motor deve ficar **morno**, não quente a ponto de não dar para encostar.

3.5 DRV8825 × A4988 (resumo das diferenças)

	DRV8825	A4988
Micropasso máximo	1/32	1/16
Corrente (com heatsink)	~1,5 A contínua (até ~2,2 A)	~1 A contínua (até ~2 A)
Vref (Rsense 0,1 Ω)	$I \div 2$	$I \times 0,8$
Posição do potenciômetro	lado oposto ao do A4988	—

São compatíveis com o mesmo soquete, mas **confirme a orientação pelo pino EN** — não use o potenciômetro como referência, pois ele troca de lado entre os dois.

4. Ligações (pinagem e periféricos)

4.1 Tabela de pinagem

Todos os botões/chaves usam o **resistor de pull-up interno** do Arduino: **fechado = nível baixo (LOW)**. Os pinos do eixo Z (D4/D7) são reaproveitados para o display, já que o jogo usa só 2 motores.

Função no jogo	Pino Arduino	Porta no CNC Shield
STEP motor esquerdo (X)	D2	soquete do driver X (X.STEP)
DIR motor esquerdo (X)	D5	soquete do driver X (X.DIR)
STEP motor direito (Y)	D3	soquete do driver Y (Y.STEP)
DIR motor direito (Y)	D6	soquete do driver Y (Y.DIR)
ENABLE dos motores	D8	EN/GND (ativo em LOW)
Display — CLK	D4	Z.STEP (header Z.STEP/DIR)
Display — DIO	D7	Z.DIR (header Z.STEP/DIR)
Joystick esquerdo — sobe	A0	Abort
Joystick esquerdo — desce	A1	Hold
Joystick direito — sobe	A2	Resume

Joystick direito — desce	A3	CoolEn
Fim de curso X	D9	X- (header END STOPS)
Fim de curso Y	D10	Y- (header END STOPS)
Alto-falante / buzzer	D11	Z- (header END STOPS)
Botão de início	D12	SpnEn
Switch de fim de percurso	D13	SpnDir
Switch do buraco 10 (vitória)	A4	SDA (header I2C)

A pasta Imagens inclui dois guias visuais para facilitar a compreensão da pinagem do CNC Shield e sua correspondência às portas do Arduino.

4.2 Botões e chaves

Simples: **ligue cada chave entre o seu pino e o GND**. Ao fechar, o pino vai a LOW e o firmware detecta. Vale para os fins de curso, o botão de início, o switch de fim de percurso e o switch do buraco 10. Cada header tem um GND ao lado para usar como retorno.

*Usamos **joysticks** com **microswitch** e ligamos apenas **sobe/desce**. Os contatos de **esquerda/direita** não são usados.*

4.3 Display TM1637

- **Encaixe físico:** para o display caber no espaço destinado a ele, **remova os pinos-macho** que vêm soldados e **solde os fios diretamente** nos furos do display.
- **Polaridade:** este display **não tem diodo de proteção** contra inversão. **Inverter VCC/GND pode danificá-lo**. Confira duas vezes antes de energizar.
- Ligações: CLK D4 (Z.STEP), DIO D7 (Z.DIR), VCC 5V, GND GND.

4.4 Áudio

O firmware usa `tone()`, que **gera a frequência por software**. Use um **alto-falante pequeno** (melhor opção) ou um **buzzer passivo**. **Não use buzzer ativo** (com oscilador interno) — ele não reproduz as melodias. Ligue um terminal ao **D11 (pino Z-)** e o outro ao **GND**.

4.5 Sentido dos motores

O firmware já assume a **ligação padrão das bobinas** (dá para aproveitar o conector original do motor de passo). Se uma ponta subir quando deveria descer, **inverte o conector de uma das bobinas** no driver para reverter o sentido.

4.6 Fonte dos motores

Ligue a fonte no borne do shield. **12 V** atende bem (a placa aceita 12–36 V). Para dois NEMA 17 nesta aplicação, **12 V / 2–3 A já é suficiente** — não é preciso somar as correntes de pico das fases. Respeite a polaridade (+ / -).

5. Software (Arduino IDE)

5.1 Instalar a biblioteca do display (só uma vez por computador)

O firmware usa a biblioteca **TM1637** — por Avishay Orpaz (avishorp) (cabeçalho `TM1637Display.h`, classe `TM1637Display`).

*Existem várias bibliotecas “TM1637” com funções diferentes. Use **exatamente** a do Avishay Orpaz, senão o código não compila.*

Opção A — pelo zip (offline): *Sketch* *Incluir Biblioteca* *Adicionar biblioteca .ZIP...* e selecione `libraries/TM1637-avishorp.zip`.

Opção B — pela internet: *Sketch* *Incluir Biblioteca* *Gerenciar Bibliotecas...*, pesquise `tm1637` e instale a opção **“TM1637” de Avishay Orpaz**.

5.2 Abrir o sketch

Entre na pasta `EscapeBuracoAnalog_v9/` e abra **`EscapeBuracoAnalog_v9.ino`**. Devem aparecer 4 abas no topo: `EscapeBuracoAnalog_v9`, `tocarMusica`, `pitches.h` e `pitches.cpp`.

5.3 Selecionar a placa e a porta

1. Ligue o Arduino Uno pelo cabo USB.
2. **Ferramentas** **Placa** **Arduino AVR Boards** **Arduino Uno**.
3. **Ferramentas** **Porta** e selecione a porta que apareceu (`COM...` no Windows; `/dev/cu.usbmodem...` ou `/dev/ttyACM0 / ttyUSB0` no macOS/Linux).

5.4 Compilar e enviar (upload)

1. **Verificar** () para compilar — deve aparecer “Compilação concluída”.
 2. **Carregar / Upload** () para gravar. Os LEDs **TX/RX** piscam durante o envio.
 3. Aguarde “**Carregado**”.
-

6. Primeira energização (ordem segura)

1. **Sem a fonte dos motores**, ligue só o USB e faça o upload. Confirme o display mostrando `1:00` com o `:` piscando.
 2. **Ajuste o Vref** de cada driver (seção 3.4).
 3. **Só então ligue a fonte dos motores**. No primeiro *homing*, mantenha a mão perto do botão de força — a barra se move sozinha.
-

7. Personalizando o jogo

Todos os ajustes ficam no `EscapeBuracoAnalog_v9.ino` :

- **Tempo da partida:** `minutos` e `segundos` (estado inicial do jogo).
 - **Velocidade dos motores:** `VELOCIDADE_MAXIMA` (rápido) e `VELOCIDADE_MINIMA` (lento/limites). Lembre: valores em microssegundos — **menor = mais rápido**.
 - **Curso da barra:** `MAX_PONTOS` (teto) e o recuo de **700 passos** no `zerarMotores()` (posição de repouso).
 - **Tolerância de desnível** entre as pontas: `MAX_DIFERENCA_PASSOS` .
 - **Escala de passos:** `PASSOS_POR_REVOLUCAO` (ver a nota de micropasso em 3.3).
-

8. Problemas comuns

Sintoma	Causa provável	Solução
<code>TM1637Display.h: No such file or directory</code>	Biblioteca não instalada ou errada	Refaça a 5.1 com a do avishorp
Erro em <code>showNumberDecEx / setSegments</code>	Instalada uma TM1637 diferente	Remova a outra e instale a do avishorp

avrdude: ... not in sync / porta não responde	Porta errada, cabo ou placa	Confira Ferramentas Porta ; troque o cabo; reconecte
Nenhuma porta aparece	Cabo “só carga” ou driver	Use cabo de dados; em clones, instale o driver CH340
Compila e envia, mas motores não giram	Falta a fonte externa do shield	Ligue a fonte (seção 4.6)
Driver queimou ao ligar	Driver encaixado invertido	Confira a orientação pelo pino EN (seção 3.2)
Motor gira para o lado errado	Ordem das bobinas	Inverta o conector de uma bobina no driver (seção 4.5)
Motor chia/perde passos ou esquenta muito	Vref mal ajustado	Recalcule e meça o Vref (seção 3.4); use heatsink
Display não acende ou esquentou	VCC/GND invertidos	Confira a polaridade (seção 4.3) — pode ter danificado
Sem som / som errado	Buzzer ativo no lugar de passivo	Use alto-falante ou buzzer passivo (seção 4.4)

9. Segurança

- O **USB sozinho não alimenta os motores**. O shield precisa da **fonte externa**.
- **Nunca encaixe/retire drivers com a placa energizada**.
- Confira a **orientação do driver** antes de ligar (invertido = queima).
- Ligue/desligue a fonte **sem as mãos na mecânica** — a barra se move no *homing*.
- Garanta **GND comum** entre Arduino e shield (já é comum pelo encaixe).

10. Atualizar o firmware no futuro

Mesmo processo: substitua os arquivos da pasta do sketch pela nova versão, abra o `.ino` principal e repita **5.3 e 5.4** (porta upload). A biblioteca só precisa ser instalada uma vez por computador.

Créditos

Projeto Escapa Buraco — Fabricio Masutti e Leonardo Gallep (Coletivo Máquina Tudo), sob licença CC BY-NC 4.0.

Bibliotecas e trabalhos de terceiros (com licenças próprias):

- Display: **TM1637** — Avishay Orpaz (LGPL 2.1) — github.com/avishorp/TM1637
- Notas musicais (`pitches.h`): HiBit — hibit.dev
- Rotina de músicas: baseada no *arduino-songs* de Robson Couto